

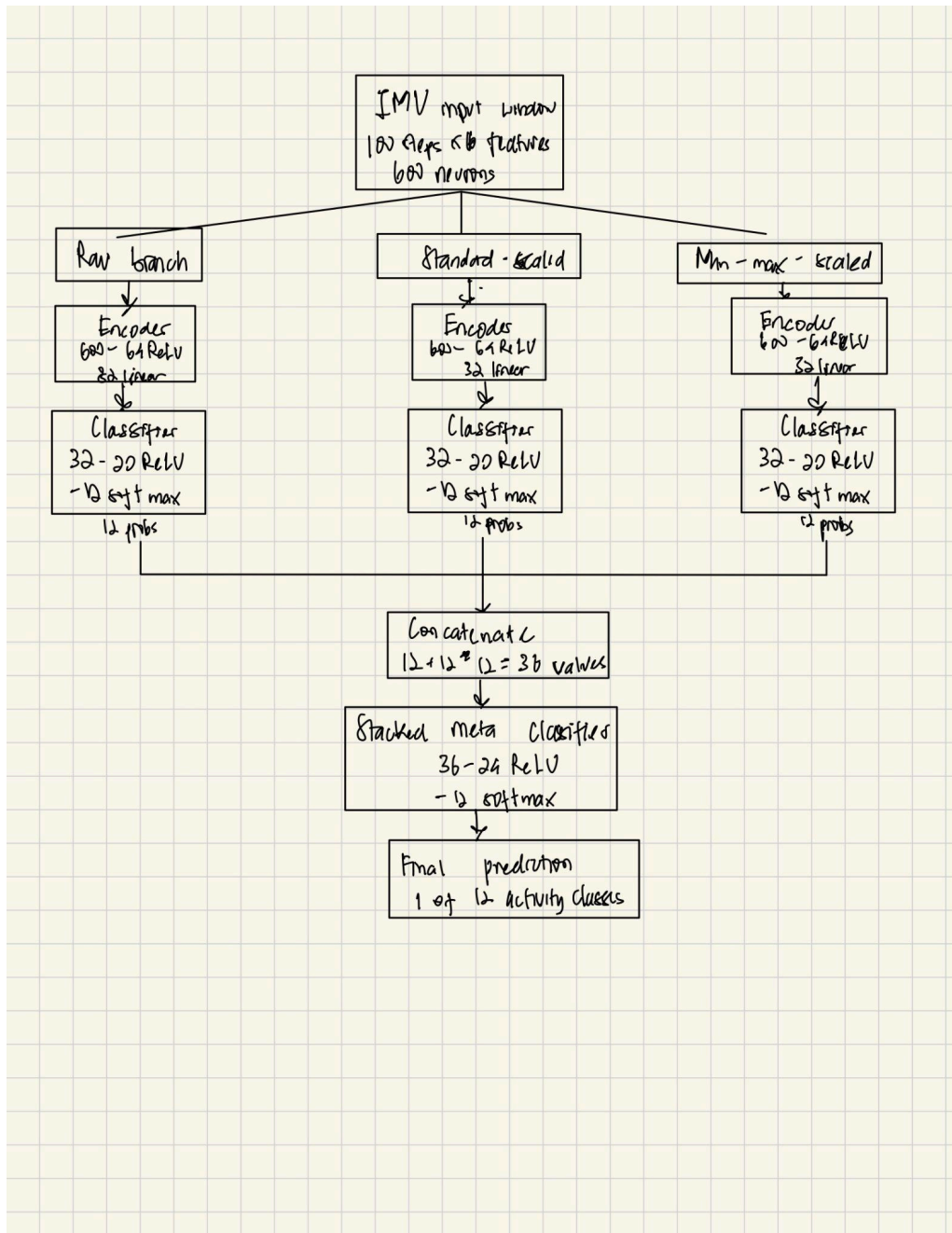
Part 1

<https://studio.edgeimpulse.com/public/1011620/live>

```
Sampling...
Timing: DSP 83 ms, inference 568 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 83 ms, inference 569 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 79 ms, inference 569 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000

Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 84 ms, inference 541 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 83 ms, inference 552 us, anomaly 0 ms, postprocessing 67 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 84 ms, inference 551 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 84 ms, inference 551 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
```

Part 2



1.

The ensemble architecture uses an encoding-decoding pipeline with 12 classes for IMU motion classification. The input data consists of 100 consecutive time steps of the 6 IMU feature signals (accelerometer x, y, z + gyroscope x, y, z), combined into a 600-neuron input layer. Three parallel pipelines are used with the same input signal, represented differently — in its raw form, as a standardized signal, and as a scaled signal — to provide diversity in each branch. All three pipelines have the same structure: the encoded data is processed by an encoder from 600 → Dense(64, ReLU) → Dense(32, linear), resulting in a 32-dimensional latent space vector, and finally, the classification stage from latent space 32 → Dense(20, ReLU) → Dense(12, Softmax) produces 12 probabilities. Concatenated together, the 12-dimensional output from all three branches

results in 36 dimensions (12 + 12 + 12), which are further classified with a stacked classifier 36 → Dense(24, ReLU) → Dense(12, Softmax).

2. First, each branch model is pruned using `prune_low_magnitude` with `PolynomialDecay` that increases sparsity from 10% to 80% in steps 0 to 500, pruning out the smallest magnitude weights. Then the pruning wrappers are stripped using `strip_pruning`, producing an unpruned Keras model that retains the sparse architecture. Pruning wrappers must be stripped before quantization, as the quantization API expects bare layer models, whereas pruning requires a special wrapper for each layer. Then the sparse model is passed to `quantize_model`, where fake-quantization operations are added to simulate int8 rounding during training, and the model is further trained to handle the reduced precision introduced by the simulation. Since pruning in combination with regular QAT will make the pruned weights non-zero, a custom callback, `MaskEnforcerCallback`, is needed to save binary masks for all pruned weights and re-mask them after every batch and epoch, ensuring sparsity throughout the whole QAT process. The model is finally exported as int8 TFLite using conversion with default optimization level and int8 input/output tensor types, and the conversion tool receives a set of about 200 training examples for calculating the proper activation range, scale, and zero point — otherwise, quantization calibration would fail, and the accuracy would drop drastically. Thus, pruning and int8 quantization together decrease the model size and memory usage 4 times (8-bit vs. 32-bit), allowing the model to use integer arithmetic and operate much faster on the Cortex-M4 CPU of the Nano 33 BLE Sense.
3. The predictions changed drastically although the board was lying flat on the table and not moving at all. Three consecutive windows produced "Waist bends forward" (0.43), "Lying down" (0.27) and "Jump front and back" (0.61). Clearly, this cannot be acceptable behavior from a static board. For a resting device we should expect only one output (with the highest possible confidence score) predicting that the board isn't moving, not changing between entirely unrelated actions and failing to provide any decent confidence score. The major problems which I found included changes in predictions between different time windows while nothing was moving, assigning active motion classes (jumping and bending waist) when the board didn't move at all, keeping low confidence scores (so that the model isn't sure what class is predicted) and disagreement between the three branches (the raw output predicted class 11 while the standard and min-max branches predicted class 1 and 4 correspondingly). There are two reasons why this is happening: first, the training data does not include an activity class for "resting", so that for every input the model needs to find some activity to attribute to, and second, in case of a motionless board, the model randomly picks one activity. To address this problem, I would recommend including a class called "rest" in the training data, which includes inputs collected on the actual board in its real operating state, and introducing a confidence score threshold alongside with multiwindow averaging.

```

Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.9492, 0.0000, 0.0000, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0430
Standard-scaled model scores: 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039, 0.0000
Min-max-scaled model scores: 0.0000, 0.2344, 0.0195, 0.1172, 0.2539, 0.0195, 0.2344, 0.1172, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.0039, 0.0078, 0.0195, 0.0117, 0.1836, 0.4297, 0.0000, 0.0039, 0.0000, 0.0898, 0.0039, 0.2500
-----
Final prediction: Waist bends forward
Class index: 5
Confidence score: 0.4297

```

```

Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0898, 0.0000, 0.0312, 0.0000, 0.2266, 0.0000, 0.0000, 0.0703, 0.0781, 0.5039
Standard-scaled model scores: 0.0000, 0.0039, 0.1641, 0.0586, 0.0000, 0.0000, 0.0000, 0.0000, 0.0469, 0.1250, 0.0391, 0.5625
Min-max-scaled model scores: 0.0000, 0.0039, 0.0625, 0.2656, 0.4453, 0.0000, 0.2227, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.0195, 0.0547, 0.2734, 0.1094, 0.0078, 0.0000, 0.0000, 0.0195, 0.0703, 0.1914, 0.0078, 0.2422
-----
Final prediction: Lying down
Class index: 2
Confidence score: 0.2734

```

```

Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.1289, 0.0000, 0.0195, 0.0000, 0.0039, 0.0000, 0.0000, 0.0234, 0.2383, 0.5859
Standard-scaled model scores: 0.0000, 0.7187, 0.0742, 0.0273, 0.0000, 0.0039, 0.0000, 0.0000, 0.0430, 0.0547, 0.0195, 0.0625
Min-max-scaled model scores: 0.0000, 0.0312, 0.1250, 0.0195, 0.7539, 0.0000, 0.0703, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.0508, 0.0547, 0.0742, 0.0117, 0.0391, 0.0000, 0.0000, 0.0117, 0.0195, 0.1250, 0.0039, 0.6055
-----
Final prediction: Jump front and back
Class index: 11
Confidence score: 0.6055

```

4. Yes, this behavior is quite unpredictable. Indeed, the board remained completely stationary, but for three consecutive windows, three motion-class predictions were given with the lowest possible level of confidence, from 0.27 to 0.61. The first explanation for such behavior lies in the discrepancy between the collected training dataset and the actual data collected with the Arduino. The mHealth dataset contains information from IMUs strapped to humans performing actions such as walking, jogging, and ascending/descending stairs, meaning each class indicates that a human is wearing the device and performing some kind of activity. In that respect, there will not be any class for "the sensor lying on the table." When the board does not move, the collected signal will be nothing more than noise from the gravitational force, and since this kind of data does not belong to any of the 12 classes in the training set, the model chooses the motion class that this noisy data is closest to. Moreover, there is an additional influence from the differences in sensors' parameters (different models of IMU chips, sampling rate, position, and axes direction); scalars fitted according to the training dataset transform even minor fluctuations in the data (caused by gravity force) into ranges unseen by the model before; finally, there are no motion actions from that list observed for a stationary board.